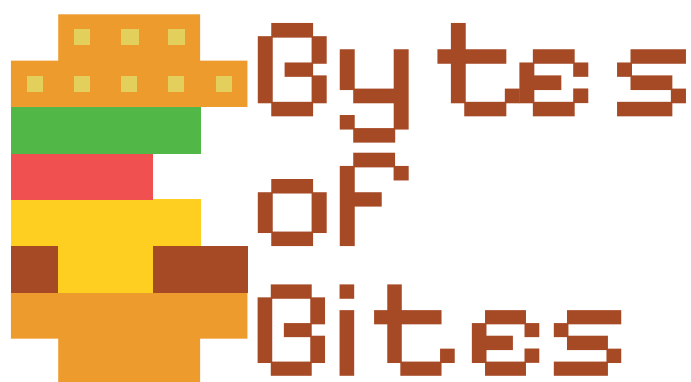


Dossier Projet :

Fil Rouge

Jalon 2



13 NOVEMBRE 2025

CDA09 – 2iSA

Créé par : Ceyhane YILMAZ



Contents

Présentation personnelle	3
Présentation de la formation CDA	4
Généralités du projet présenté (Fil rouge ou PAE)	2
Liste des compétences du référentiel couvertes par le projet	3
Expression des besoins du projet.....	4
Contexte dans lequel s'inscrit ce projet.....	4
Gestion du projet	5
Organisation et suivi de l'activité.....	5
État d'avancement	5
Analyse du besoin et spécifications fonctionnelles	6
Spécifications techniques.....	8
Présentation détaillée d'une fonctionnalité	9
Architecture technique mise en œuvre	9
Description et enchaînement des écrans	10
Illustration avec un Diagramme de Séquence	10
Sécurité de l'application.....	15
Extraits de code les plus significatifs.....	16
Plan de test	18
Plan de déploiement	19
Détail des Workflows GitHub Actions	19
Conformité et Responsabilité Numérique	20
1. Respect du Règlement Général sur la Protection des Données (RGPD).....	20
2. Principes d'Éco-conception (Green IT).....	20
Bilan.....	21
Apports et Apprentissages.....	21
Difficultés rencontrées.....	21
Perspectives	21
Annexe	23

Présentation personnelle

Je m'appelle **Ceyhane YILMAZ** et je me forme actuellement pour devenir **Concepteur Développeur d'Applications Full-Stack**.

Depuis près de dix ans, je nourris un réel intérêt pour la résolution de problèmes informatiques : automatiser des tâches, traquer des bugs, explorer de nouvelles technologies... Chaque défi m'a permis d'affiner mon intuition et d'acquérir une solide logique de développeur.

Après une première année de licence en informatique à Perpignan, j'ai choisi de réorienter mon parcours vers une approche plus concrète du développement. Cette période, marquée par la pandémie, m'a aussi permis de travailler sur des projets personnels : contribution à un jeu vidéo (*Soundsphere*) et création de scripts d'automatisation.

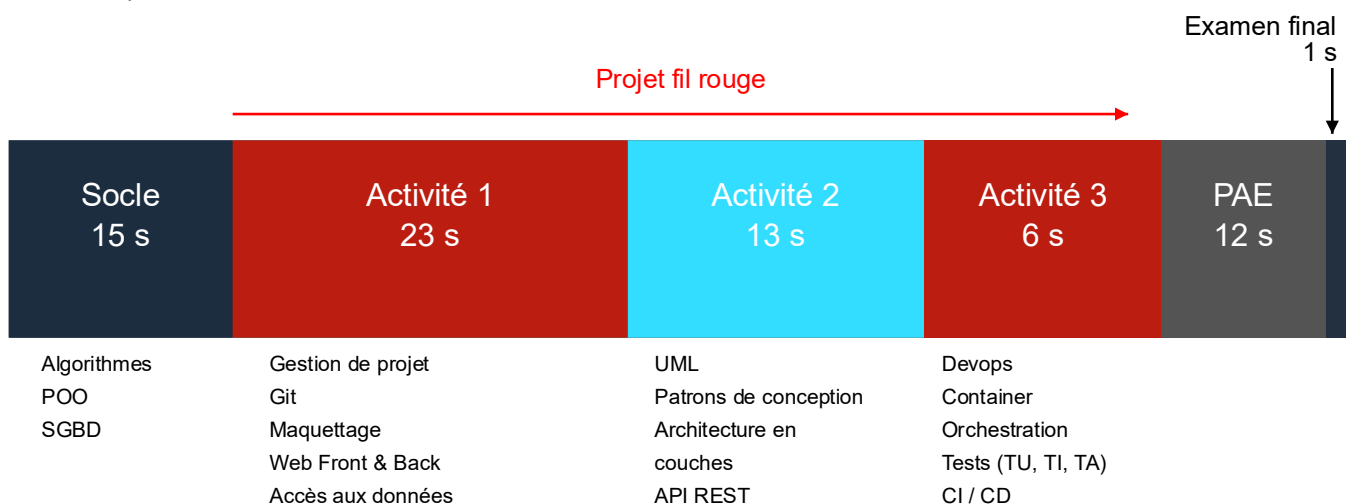
Aujourd'hui, au sein de **2iSA**, je mets à profit mes expériences pour consolider mes compétences techniques et concrétiser mes ambitions professionnelles dans le développement logiciel. Je suis motivé à transformer ma curiosité et ma rigueur en une véritable expertise.

Présentation de la formation CDA

La formation est structurée en **3 activités** de sorte à **permettre l'acquisition des compétences professionnelles** du titre Concepteur Développeur d'Applications (CDA) selon une progression pédagogique adaptée au public accueilli.

En parallèle des apprentissages, dans le but de **renforcer la pratique professionnelle**, les stagiaires mènent un projet fil rouge, **une mise en situation professionnelle** qui les amène à construire une application en plusieurs étapes.

Le parcours se déroule sur 69 semaines dont 12 semaines de Période d'Application en Entreprise (PAE) et 2 semaines passerelles.



Activité 1 – Développer une application

- Gérer son environnement de développement
- Développer une interface utilisateur
- Développer les composants une application
- Collaborer à la gestion d'un projet informatique

Activité 2 - Concevoir et développer une application organisée en couches

- Analyser les besoins et maquetter l'application
- Définir l'architecture logicielle d'une application
- Concevoir et mettre en place une base de données relationnelle
- Développer des composants d'accès aux données SQL et NoSQL

Activité 3 – Préparer le déploiement d'une application

- Préparer et documenter le déploiement d'une application
- Préparer et exécuter les plans de tests d'une application
- Contribuer à la mise en production dans une démarche DevOps

En transverse :

- Utiliser l'anglais dans son activité professionnelle
- Actualiser et partager ses compétences en développement informatique
- Communiquer dans le contexte professionnel

Généralités du projet présenté (Fil rouge ou PAE)

Le **Projet Fil Rouge** constitue la colonne vertébrale de la formation CDA. Il simule une situation réelle de développement dans laquelle je dois concevoir une application complète, du back-end au déploiement.

Objectifs Pédagogiques :

- Il vise à **apporter de la cohérence aux apprentissages** en mobilisant l'ensemble des compétences acquises au fil des modules.
- Il sert de support principal pour les **Évaluations Passées en Cours de Formation (EPCF)**, permettant aux étudiants de démontrer l'acquisition des compétences requises pour le titre professionnel.
- Il permet de **mettre en œuvre les compétences essentielles** associées au module Déploiement.
- Il contribue à l'alimentation du **Dossier Professionnel (DP)**.

Structure :

Le projet est divisé en **deux jalons principaux** avant l'évaluation finale (PAE), chacun avec des livrables spécifiques et donnant lieu à une évaluation.

- **Jalon 1** : Livraison de l'application web ciblant un sous-ensemble de fonctionnalités.
- **Jalon 2 (actuel)** : Développement d'une application desktop, API REST, plan de tests et déploiement.

Pour ce **Jalon 2**, le travail consiste à concevoir une API REST sécurisée en ASP.NET Core et une application desktop WinForms qui consomme cette API. Ce projet s'inscrit dans le cadre de l'entreprise fictive *Gastronome Créatif*. L'application, que j'ai baptisée *Bytes of Bites*, a pour but de faciliter la gestion et la consultation de recettes.

Liste des compétences du référentiel couvertes par le projet

Ce second jalon me permet de mettre en œuvre et de faire évaluer les compétences suivantes du **Référentiel Emploi Activités Compétences (REAC)** :

- **Concevoir et développer une application sécurisée organisée en couches**
 - Analyser les besoins et maquetter une application
 - Définir l'architecture logicielle d'une application
 - Concevoir et mettre en place une base de données relationnelle
 - Développer des composants d'accès aux données SQL et NoSQL
- **Préparer le déploiement d'une application sécurisée**
 - Préparer et exécuter les plans de test d'une application
 - Préparer et documenter le déploiement d'une application
 - Contribuer à la mise en production dans une démarche DevOps
- **Compétences transverses**
 - Actualiser et partager ses compétences en conception et développement

Expression des besoins du projet

Contexte dans lequel s'inscrit ce projet

Ce projet s'inscrit dans le cadre de ma formation de CDA et constitue le second jalon du Projet Fil Rouge. Son but premier est pédagogique.

- **Mise en Situation Professionnelle** : Il simule un contexte professionnel où je dois développer une API REST et une application Desktop Winforms pour un client fictif (*Gastronome Créatif*).
- **Application des Compétences** : L'objectif est de mettre en œuvre les compétences acquises durant la formation, notamment celles liées à l'architecture en couches, au développement d'API, à la conception d'une application « client lourd » (desktop) et à la préparation du déploiement.
- **Évaluation Formative** : Le projet sert de support principal à l'EPCF. Mon travail sur ce jalon sera évalué pour démontrer l'acquisition des compétences des activités 2 (« Concevoir et développer une application sécurisée organisée en couches ») et 3 (« Préparer le déploiement d'une application sécurisée »).
- **Constitution du Dossier Professionnel** : Les réalisations (code, documents de conception, plans de test et de déploiement) sont destinées à alimenter mon Dossier Professionnel (DP).

Gestion du projet

La gestion de projet est une compétence évaluée. Pour ce jalon, elle a impliqué :

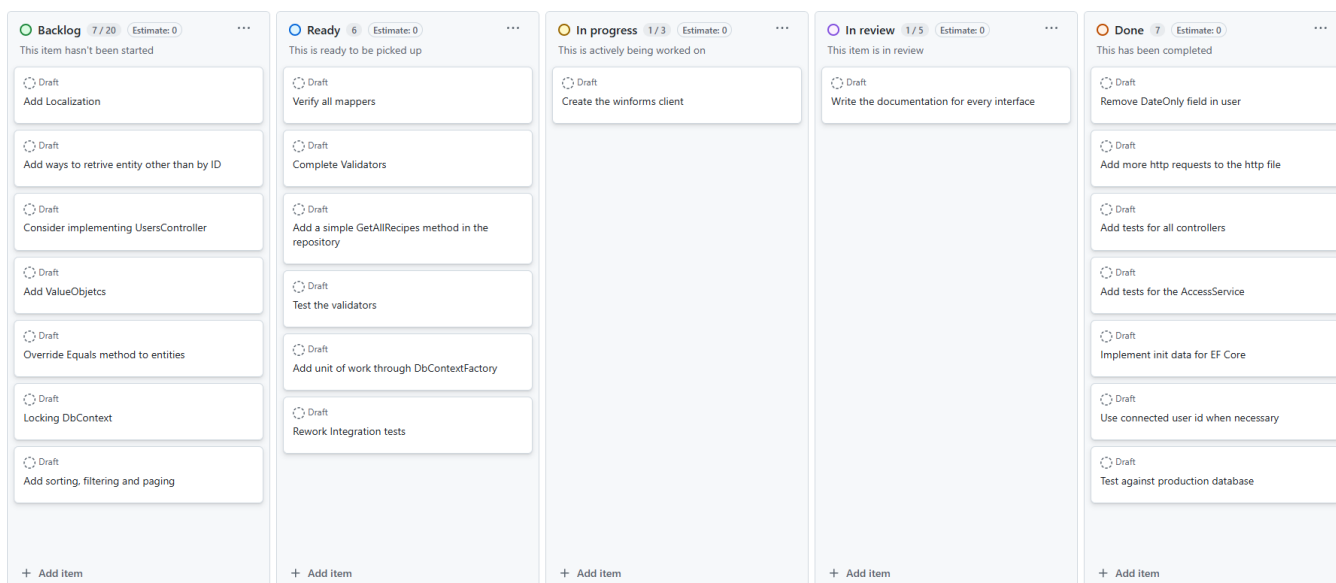
- Planification et suivi des tâches.
- Utilisation d'outils collaboratifs (GitHub Projects).
- Communication au sein de l'équipe pédagogique (jouant le rôle de Maîtrise d'Ouvrage) via Teams.

Organisation et suivi de l'activité

La gestion de ce projet fil rouge, compétence clé évaluée (CP4 du REAC), a débuté par une phase d'exploration du Framework WinForms et de l'architecture multicouches de l'API REST, dont je reste familier puisque similaire au MVC du jalon 1. Conscient de la nécessité d'un suivi structuré, j'ai alimenté un tableau Kanban sur GitHub Projects. Cet outil m'a permis de visualiser les tâches selon leur état (Backlog, Ready, In Progress, In Review, Done), facilitant le suivi de l'avancement et l'organisation du travail. La communication avec les formateurs a également été un élément de suivi constant.

État d'avancement

Suivre l'avancement du projet se fait sur l'onglet « Projects » de la plateforme **GitHub** et se présente comme sur l'image ci-dessous :



Au moment de ce jalon, les livrables suivants sont réalisés :

- L'API REST est fonctionnelle, avec les endpoints CRUD (Create, Read, Update, Delete) pour les recettes, les ingrédients et les catégories.
- L'authentification est sécurisée via des JSON Web Tokens (JWT).
- L'application Desktop WinForms est développée et se connecte avec succès à l'API.
- L'API et la base de données PostgreSQL sont conteneurisées (dockerisées).
- Les jeux de tests unitaires et d'intégration sont écrits et s'exécutent automatiquement via GitHub Actions.

Analyse du besoin et spécifications fonctionnelles

L'analyse du cahier des charges a permis de définir les acteurs et les fonctionnalités attendues pour ce Jalon 2.

Acteurs

- **Utilisateur** : Doit être authentifié. Peut consulter la liste des recettes et le détail des ingrédients pour une recette sélectionnée.
- **Administrateur** : Dispose des mêmes droits que l'utilisateur. De plus, il peut gérer les catégories de recettes (ajout, modification, suppression) et les associations entre les recettes et les catégories.

Fonctions principales

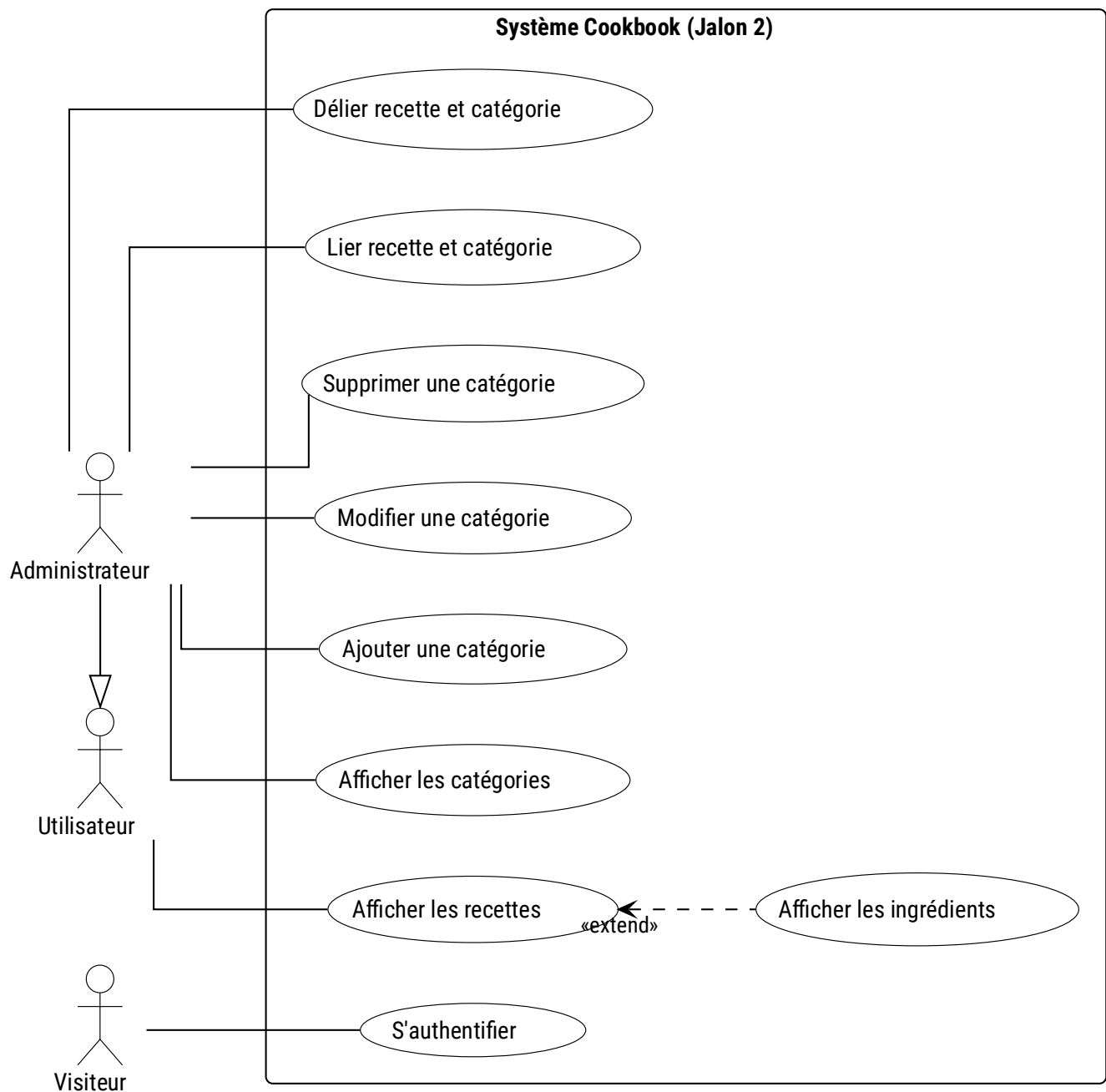
1. **Authentification** : L'accès à l'application desktop est conditionné par une authentification sécurisée (JWT).
2. **Consultation (Utilisateur/Admin)** : Affichage de la liste des recettes et consultation des ingrédients d'une recette.
3. **Gestion (Admin)** : Gestion complète (CRUD) des catégories et gestion des liens recettes/catégories.

Règles de gestion

- L'authentification est obligatoire pour accéder à toute fonctionnalité de l'application.
- En cas d'échec d'authentification, un message clair doit en informer l'utilisateur.
- La suppression d'une catégorie n'est autorisée que si plus aucune recette n'y est associée.

Cette analyse permet de construire ce diagramme de cas d'utilisation :

Diagramme des Cas d'Utilisation - Jalon 2 (API + WinForm)

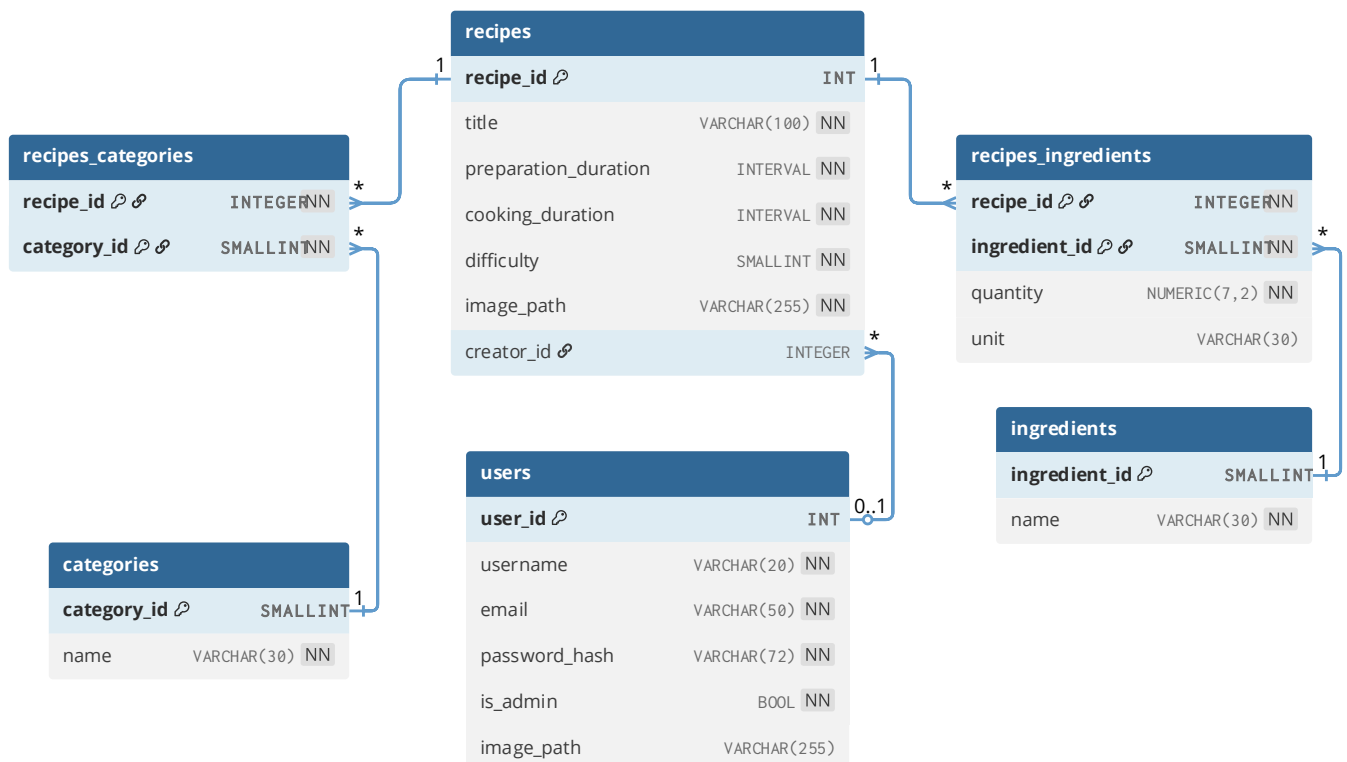


Spécifications techniques

Pour répondre aux exigences techniques du cahier des charges, j'ai mis en place un environnement de développement cohérent. L'architecture repose sur des technologies modernes et robustes issues de l'écosystème .NET.

Élément	Technologie / Outil
Langage principal	C#
Framework Back-End	ASP.NET Core Web API
Framework Front-End (desktop)	WinForms
Base de données	PostgreSQL (container Docker)
ORM	Entity Framework Core (EF Core)
Authentification	JSON Web Tokens (JWT)
IDE	Visual Studio / JetBrains Rider
Contrôle de version	Git & GitHub
Conteneurisation	Docker
CI/CD	GitHub Actions
Modélisation	UML pour l'API, maquettes directes pour WinForms

(Note : J'ai opté pour EF Core pour l'accès aux données de l'API, mais le diagramme de la base de données reste valable.)



Présentation détaillée d'une fonctionnalité

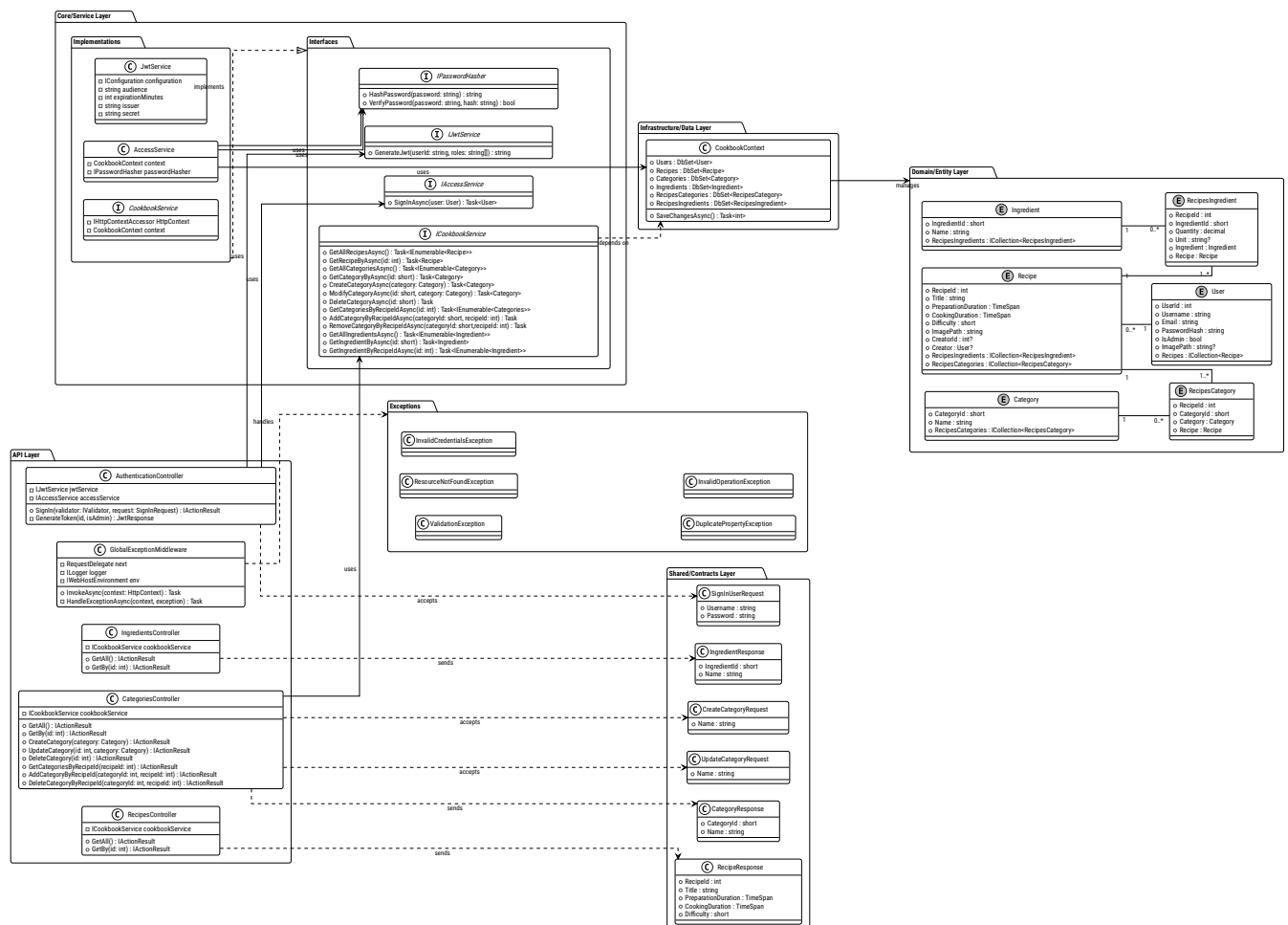
Architecture technique mise en œuvre

J'ai structuré l'API REST selon une architecture en couches pour garantir la séparation des responsabilités, la maintenabilité et la testabilité du code.

1. **Couche API (Présentation)** : Gérée par les Controllers (ex: CategoriesController). Son rôle est de recevoir les requêtes HTTP, de valider les entrées et de retourner les réponses (JSON). Elle gère par exemple l'authentification et l'autorisation.
2. **Couche Métier (BLL - Business Logic Layer)** : Gérée par les Services (ex: CookbookService). C'est ici que réside la logique applicative et les règles de gestion (par exemple, "vérifier qu'une recette a toujours au moins une catégorie avant suppression").
3. **Couche d'Accès aux Données (DAL - Data Access Layer)** : Gérée par Entity Framework Core (CookbookContext). Son rôle est de traduire les appels de la couche métier en requêtes SQL vers la base de données PostgreSQL.

Ce découplage est essentiel. Par exemple, l'application WinForms (le client) communique uniquement avec la couche API, qui elle-même ne connaît que la couche métier (BLL), et seule la BLL dialogue avec la DAL.

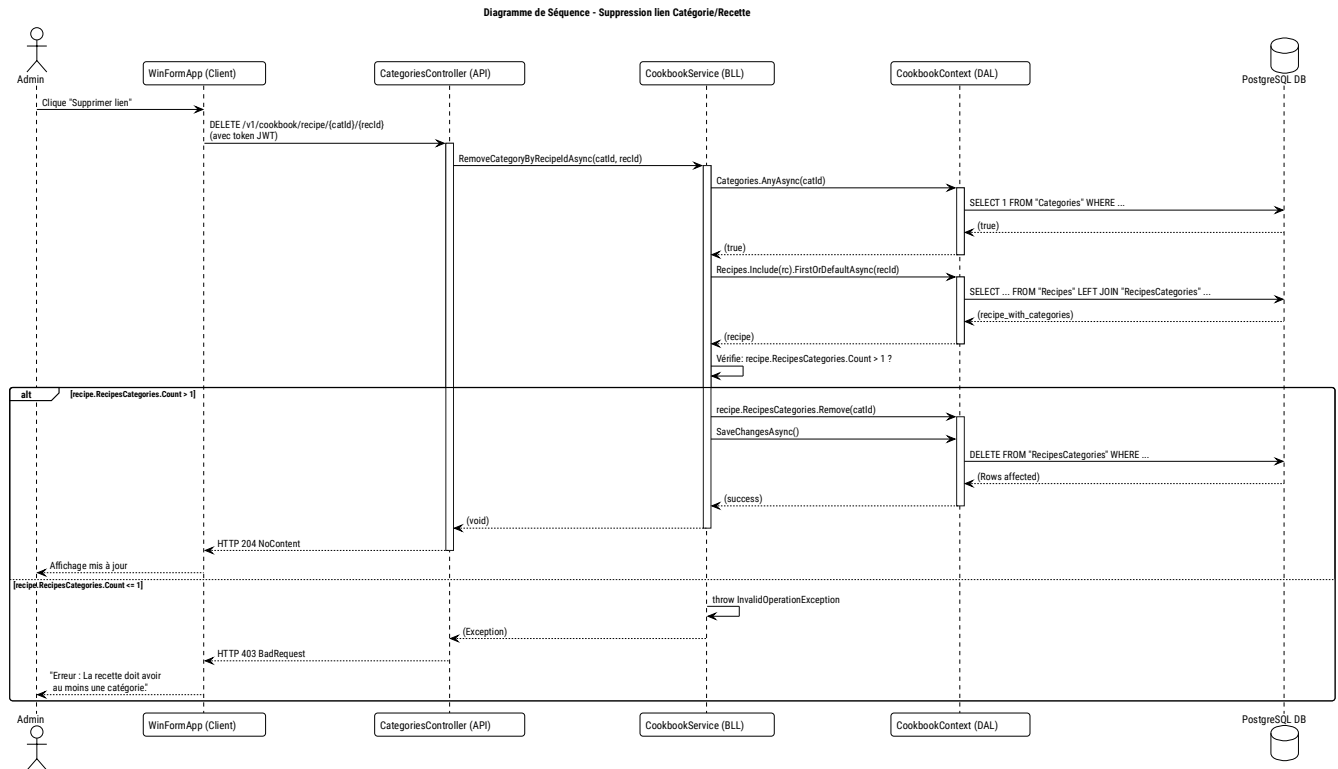
On peut alors construire le diagramme de classe suivant :



Description et enchaînement des écrans

Illustration avec un Diagramme de Séquence

Pour illustrer concrètement la collaboration entre ces couches, j'ai modélisé un diagramme de séquence pour le cas d'utilisation "Supprimer un lien entre une recette et une catégorie".



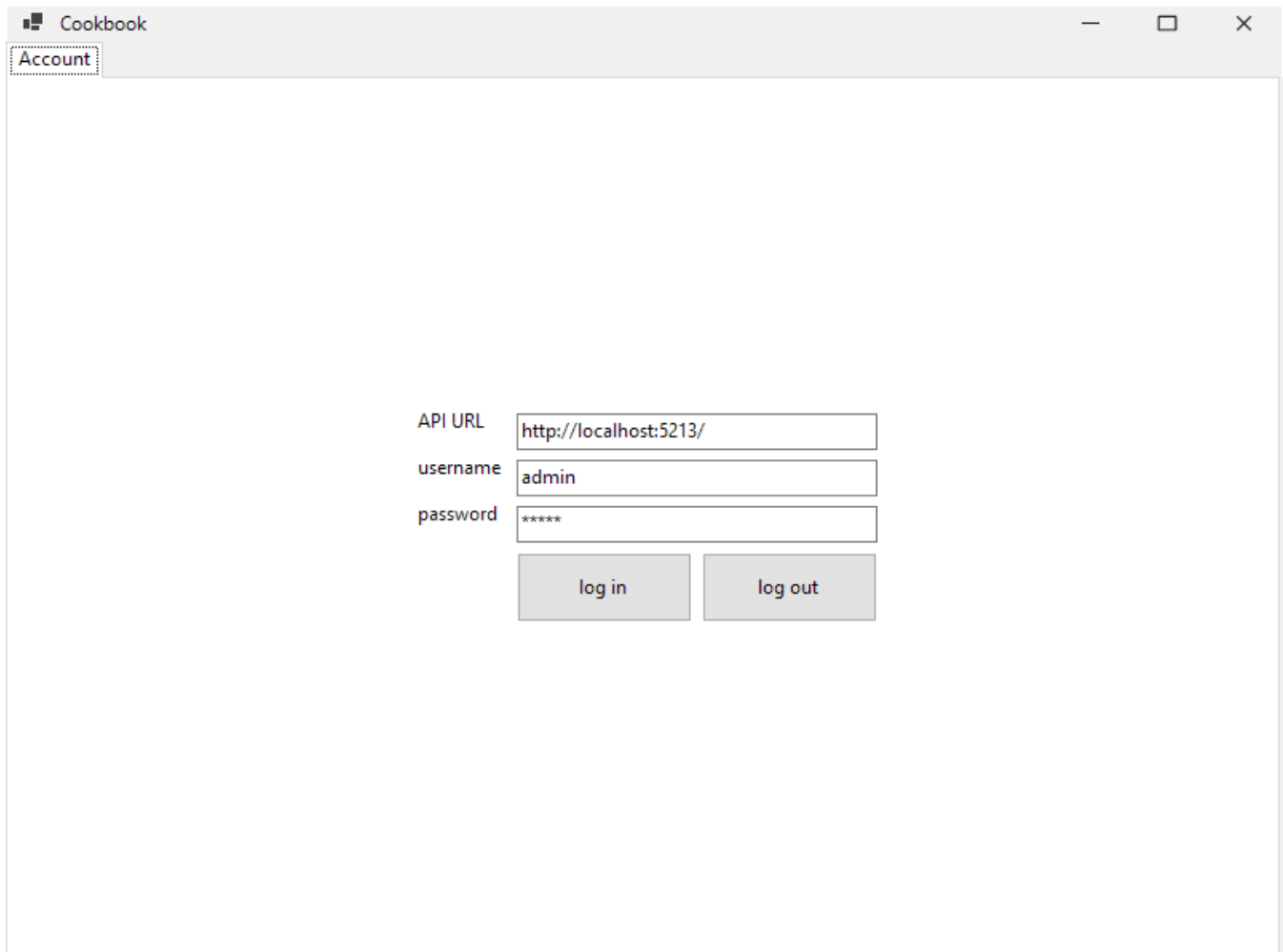
L'administrateur est connecté et se trouve sur l'onglet de gestion des catégories de recettes.

Ce diagramme montre le flux suivant :

1. L'**Admin** initie l'action depuis l'application **WinForm**.
2. Le **WinForm** envoie une requête DELETE au CategoriesController (Couche API), en y joignant le token JWT d'authentification.
3. Le **Controller** (Couche API) valide la requête, vérifie l'autorisation (rôle "admin") et appelle la méthode RemoveCategoryByRecipeldAsync du CookbookService (Couche BLL).
4. Le **Service** (Couche BLL) prend le relais. Il demande d'abord au CookbookContext (Couche DAL) de charger la recette concernée ainsi que la liste de ses catégories associées.
5. **Point Clé (Règle de gestion)** : Arrivé à ce point, le Service applique la logique métier. Il vérifie si le nombre de catégories associées à la recette est supérieur à 1.
6. **Si oui**, il demande au Context (DAL) de supprimer le lien et d'exécuter SaveChangesAsync. L'opération réussit et une réponse positive est renvoyée.
7. **Si non** (c'est la dernière catégorie), le Service lève une InvalidOperationException. Cette exception est interceptée et l'API retourne une erreur, empêchant ainsi la suppression et garantissant le respect du cahier des charges.

Voici la description du scénario utilisateur correspondant aux captures d'écran de l'application WinForms :

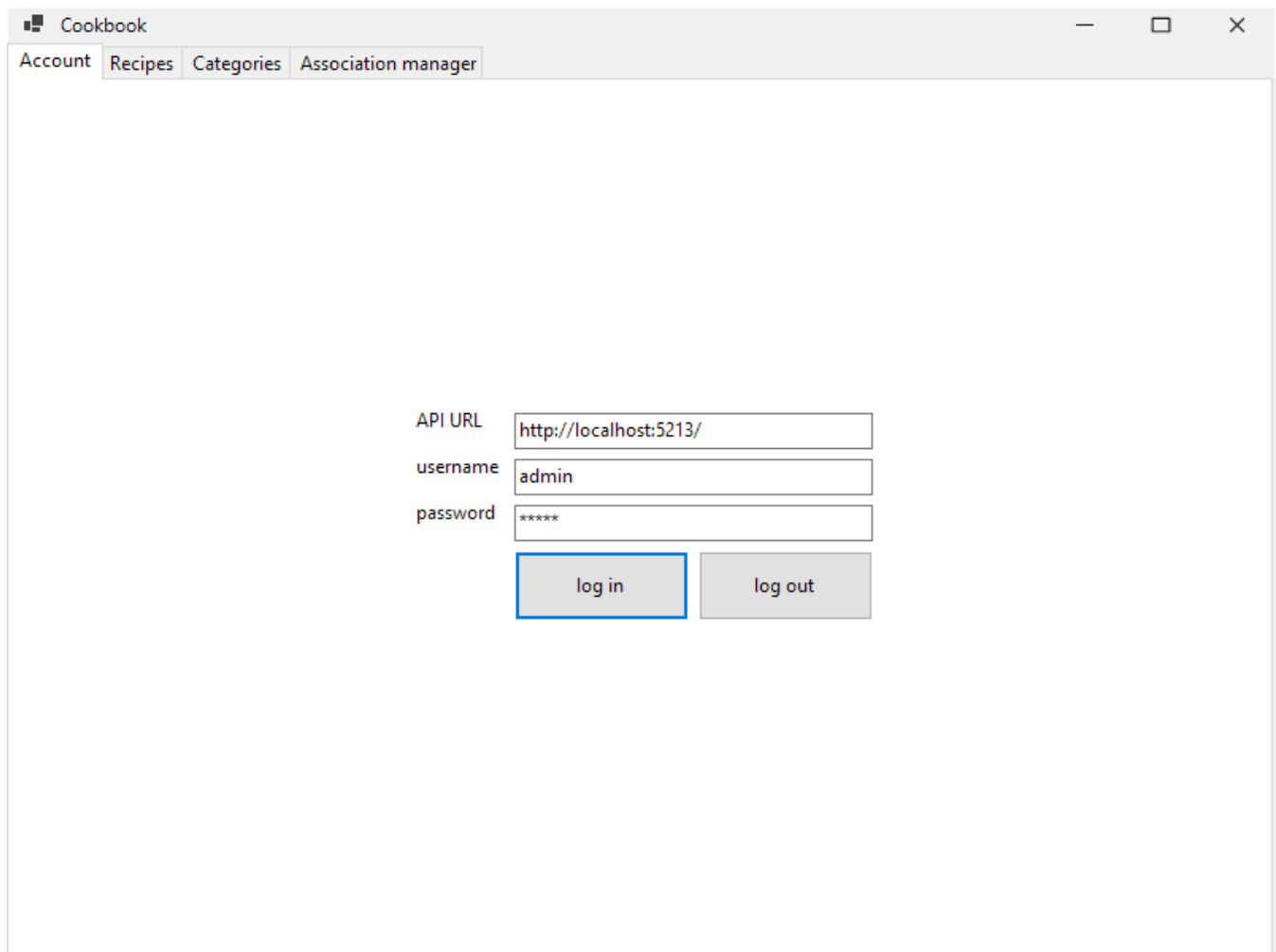
Connexion : Au lancement, l'application s'ouvre sur l'onglet « Account ». Je dois d'abord renseigner l'URL de l'API (ici, pointant vers localhost où s'exécute mon conteneur Docker), puis mon nom d'utilisateur ("admin") et mon mot de passe.



The screenshot shows a Windows application window titled "Cookbook". Inside the window, there is a tab labeled "Account". The main content area of the window contains a login form. The form has three input fields: "API URL" with the value "http://localhost:5213/", "username" with the value "admin", and "password" with the value "*****". Below these fields are two buttons: "log in" and "log out".

API URL	http://localhost:5213/
username	admin
password	*****
log in log out	

Accès aux fonctionnalités : Après une connexion réussie, l'API me renvoie un token JWT. L'application WinForms le stocke et déverrouille l'accès aux autres onglets : "Recipes", "Categories", et "Association manager".



The screenshot shows a Windows application window titled "Cookbook". It has a tabbed interface with four tabs: "Account", "Recipes", "Categories", and "Association manager". The "Account" tab is currently selected. The main content area of the "Account" tab contains a login form with the following elements:

- A label "API URL" followed by a text input field containing "http://localhost:5213/".
- A label "username" followed by a text input field containing "admin".
- A label "password" followed by a text input field containing "*****".
- Two buttons: "log in" (highlighted with a blue border) and "log out".

Gestion des associations : L'onglet "Association manager" est l'écran principal pour la gestion.

- Le panneau de gauche liste toutes les recettes.
- Le panneau en bas à droite liste toutes les catégories disponibles.
- Le panneau en haut à droite affiche les catégories *actuellement associées* à la recette sélectionnée (ici, "The salad").

Les boutons "add" et "remove" permettent à l'administrateur de gérer ces associations.

Cookbook

Account Recipes Categories **Association manager**

	Title
▶	The Salad
	The Salad 2
	Simple Pasta Ba...
	Quick Omelette
	Chocolate Chip...
	Classic Beef Bur...
	Vegetable Stir-Fry
	Lemonade
	Chicken Noodl...
	Guacamole
	Pancakes
*	

	Name
▶	Salad
*	

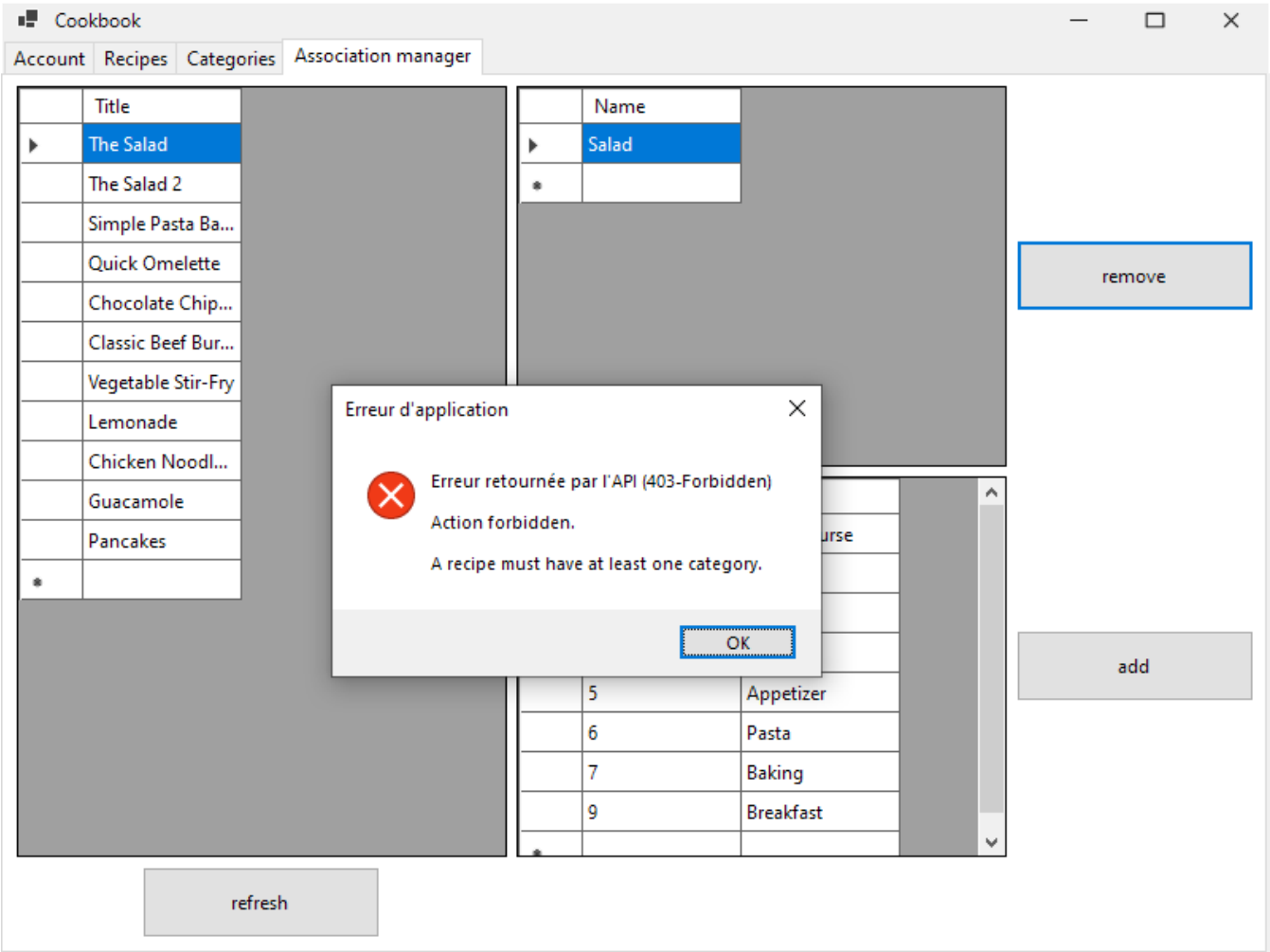
remove

	CategoryId	Name
▶	1	Main Course
	2	Side dish
	3	Dessert
	4	Drink
	5	Appetizer
	6	Pasta
	7	Baking
	9	Breakfast
*		

add

refresh

Application de la règle de gestion : Cette capture d'écran est la preuve de l'application de la règle de gestion. J'ai sélectionné l'unique catégorie ("Main Course") de la recette "The salad" et cliqué sur "remove". L'API a correctement refusé l'opération (en suivant la logique du diagramme de séquence) et a renvoyé une erreur. L'application WinForms a intercepté cette erreur et affiche le message renvoyé par l'API : "a recipe must have at least one category".



Sécurité de l'application

J'ai mis en œuvre une authentification JWT pour sécuriser l'accès à l'API. Chaque utilisateur reçoit un token signé lors de sa connexion, valable pour une durée limitée.

Les rôles (utilisateur / administrateur) sont intégrés dans le token. J'utilise ensuite les attributs de .NET pour protéger les endpoints :

- `[Authorize(Roles = "admin,user")]` est utilisé pour les actions de consultation (ex: voir les recettes).
- `[Authorize(Roles = "admin")]` est utilisé pour les actions sensibles (ex: supprimer une catégorie).

Des tests de sécurité ont été réalisés pour vérifier la gestion des erreurs d'authentification, la protection contre l'accès non autorisé (ex: un simple "user" tentant d'accéder à une route "admin") et la validation des entrées.

Extraits de code les plus significatifs

Pour illustrer l'architecture et la sécurité, voici deux extraits de code représentatifs. Le premier est un *endpoint* du contrôleur (Couche API) et le second est la méthode de service correspondante (Couche BLL). Cette fonctionnalité permet de supprimer l'association entre une catégorie et une recette.

1. CategoriesController.cs (Couche API)

```
1 using Cookbook.Core;
2 using Cookbook.SharedData.Contracts.Requests;
3 using Cookbook.SharedData.Mappers;
4 using FluentValidation;
5 using Microsoft.AspNetCore.Authorization;
6 using Microsoft.AspNetCore.Mvc;
7
8 namespace Cookbook.API.Controllers;
9
10 [Authorize(Roles = "admin,user")]
11 [Route("v1/cookbook/[controller]")]
12 [ApiController]
13 public class CategoriesController(ICookbookService cookbookService) : ControllerBase
14 {
15     [HttpDelete("recipe/{categoryId:int}/{recipeId:int}")]
16     [ProducesResponseType(StatusCodes.Status200OK)]
17     [ProducesResponseType(StatusCodes.Status404NotFound)]
18     [ProducesResponseType(StatusCodes.Status500InternalServerError)]
19     public async Task<IActionResult> RemoveCategoryByRecipeId(int categoryId, int recipeId)
20     {
21         await cookbookService.RemoveCategoryByRecipeIdAsync((short)categoryId, recipeId);
22         return NoContent();
23     }
24 }
```

Cet extrait montre un endpoint sécurisé qui prend les identifiants de la recette et de la catégorie en paramètre et appelle le service métier. Il ne contient aucune logique, seulement la gestion de la requête HTTP.

2. CookbookService.cs (Couche BLL)

```
1 using System.Security.Claims;
2 using Microsoft.AspNetCore.Http;
3 using Cookbook.Infrastructure;
4 using Cookbook.SharedData.Entities;
5 using Cookbook.SharedData.Exceptions;
6 using Microsoft.EntityFrameworkCore;
7
8 namespace Cookbook.Core;
9
10 public class CookbookService(
11     IHttpContextAccessor httpContextAccessor,
12     CookbookContext context) : ICookbookService
13 {
14     public async Task RemoveCategoryByRecipeIdAsync(short categoryId, int recipeId)
15     {
16         var categoryExists = await context.Categories
17             .AnyAsync(c => c.CategoryId == categoryId);
18
19         var recipe = await context.Recipes
20             .Where(r => r.RecipeId == recipeId)
21             .Include(r => r.RecipesCategories)
22             .FirstOrDefaultAsync();
23         ?? throw new ResourceNotFoundException(typeof(Recipe), nameof(Recipe), recipeId);
24
25         if (!categoryExists)
26             throw new ResourceNotFoundException(typeof(Category), nameof(Category), categoryId);
27
28         if (recipe.RecipesCategories.Count > 1)
29         {
30             var recipeCategoryToRemove = recipe.RecipesCategories
31                 .FirstOrDefault(rc => rc.CategoryId == categoryId);
32             if (recipeCategoryToRemove is not null)
33             {
34                 recipe.RecipesCategories.Remove(recipeCategoryToRemove);
35                 await context.SaveChangesAsync();
36             }
37             else
38                 throw new ResourceNotFoundException(typeof(RecipeCategory),
39                     nameof(RecipeCategory), (recipeId, categoryId));
40         }
41         throw new InvalidOperationException("A recipe must have at least one category.");
42     }
43 }
```

Cet extrait illustre la couche BLL. Il orchestre la DAL (via context) et applique des règles de gestion (vérifier l'existence, vérifier le nombre de catégories restantes) avant d'autoriser la suppression.

Plan de test

La préparation et l'exécution des plans de test sont une compétence clé de l'Activité 3. Pour garantir la qualité et la non-régression de l'API, j'ai mis en place une stratégie de tests automatisés.

J'ai utilisé **xUnit** pour le framework de test et **Testcontainers** pour créer à la volée une base de données PostgreSQL éphémère (en Docker) pour les tests d'intégration, assurant ainsi un environnement de test propre et isolé. Ces tests sont exécutés automatiquement à chaque *push* et *pull request* sur GitHub grâce à GitHub Actions.

Ma stratégie de test s'articule en deux volets :

1. Tests Unitaires : J'ai ciblé la logique métier pure, en isolant les services de la base de données (via des mocks) et des contrôleurs. Les tests unitaires vérifient des points précis comme :
 - PasswordHasherService : S'assurer que le hachage et la vérification des mots de passe fonctionnent (ex: `VerifyPassword_WithCorrectPassword_ReturnsTrue`, `HashPassword_ProducesConsistentLength`).
 - CookbookService : Valider les règles de gestion (ex: `DeleteCategoryAsync_WithNonExistingCategory_ThrowsResourceNotFoundException`, `ModifyRecipeAsync_WithValidRecipe_UpdatesAndReturnsRecipe`).
 - AuthenticationController : Vérifier la logique d'authentification (ex: `SignIn_WithInvalidCredentials_ThrowsInvalidCredentialsException`, `SignUp_WithAdminUser_ReturnsTokenWithAdminAndUserRoles`).
2. **Tests d'Intégration** : Ces tests valident le comportement complet de l'application, de la requête HTTP jusqu'à la réponse de la base de données (via Testcontainers). J'ai testé des workflows entiers :
 - RecipesController : Un test complet (`CompleteWorkflow_CreateReadUpdateDelete_Success`) qui crée une recette (en tant qu'admin), la lit, la modifie et la supprime, en vérifiant les codes HTTP et les données à chaque étape.
 - AccessService / AuthenticationController : Des tests qui simulent une inscription (`SignUpAsync_WithValidUser_CreatesUserSuccessfully`) suivie d'une connexion (`SignInAsync_WithValidCredentials_ReturnsUser`), ou qui vérifient la gestion des doublons (ex: `SignUp_WithDuplicateEmail_ThrowsDuplicatePropertyException`).
 - Sécurité : Des tests vérifient que l'accès est refusé sans authentification (`GetAll_WhenNotAuthenticated_ReturnsUnauthorized`) ou avec un rôle insuffisant (`Create_AsRegularUser_ReturnsForbidden`).

Plan de déploiement

La dernière étape de ce jalon consistait à préparer le déploiement de l'application (Activité 3).

Détail des Workflows GitHub Actions

1. **dotnet.yml (Intégration Continue - CI)**

- **Nom** : Cookbook API build
- **Déclencheur** : Ce workflow se lance à chaque push ou pull_request sur la branche master.
- **Objectif** : Valider la qualité du code avant la fusion.
- **Étapes** : Il installe la version preview de .NET 10, restaure les dépendances (dotnet restore), compile le projet en mode Release (dotnet build), et, étape cruciale, il exécute l'ensemble des tests (dotnet test). Si un seul test échoue, le pipeline est marqué comme "échoué", bloquant une éventuelle fusion défectueuse.

2. **docker-image.yml (Déploiement Continu - CD)**

- **Nom** : Docker
- **Déclencheur** : Ce workflow, plus sensible, ne se lance que lors d'un push sur la branche release ou lors de la création d'un tag de version (ex: v1.0.0).
- **Objectif** : Construire et publier l'application sous forme d'image Docker.
- **Étapes** :
 - Il se connecte à la registry de conteneurs de GitHub (ghcr.io) en utilisant un GITHUB_TOKEN sécurisé.
 - Il utilise docker/setup-buildx-action pour préparer l'environnement de build.
 - Il extrait les métadonnées pour taguer l'image (ex: ghcr.io/ceynou/cookbook:latest ou ghcr.io/ceynou/cookbook:v1.0.0).
 - Il lance docker/build-push-action qui lit le Dockerfile de l'API, construit l'image, et la pousse (push) vers la registry ghcr.io.
 - Enfin, il utilise cosign pour signer l'image, garantissant ainsi son authenticité et son intégrité.

Ce pipeline CI/CD automatise entièrement le processus, des tests à la publication, ce qui constitue le cœur de la compétence "Contribuer à la mise en production dans une démarche DevOps".

J'ai conteneurisé l'API et la base de données PostgreSQL à l'aide de **Docker Compose**. Mon script docker-compose.yml définit les services :

- Un conteneur pour la base de données PostgreSQL, qui utilise un volume Docker persistant pour que les données ne soient pas perdues au redémarrage.
- Un conteneur pour l'API REST, construit à partir d'un Dockerfile, qui expose le port 8080 pour l'application WinForms.
- Un réseau Docker privé pour que l'API et la base de données communiquent de manière sécurisée et isolée.

De plus, j'ai configuré un pipeline **GitHub Actions** pour l'intégration et le déploiement continu (CI/CD) :

1. **CI (Intégration Continue)** : À chaque *push*, le pipeline compile le code et exécute l'intégralité des tests unitaires et d'intégration.
2. **CD (Déploiement Continu)** : Si les tests réussissent, le pipeline construit l'image Docker de l'API et la publie automatiquement sur la registry privée de GitHub Packages.

Conformité et Responsabilité Numérique

En tant que Concepteur Développeur d'Applications, ma responsabilité ne se limite pas à produire du code fonctionnel. Elle s'étend à la protection des utilisateurs et à l'impact environnemental de mes applications. J'ai pris en compte ces deux piliers, le RGPD et l'Éco-conception, dans ce projet.

1. Respect du Règlement Général sur la Protection des Données (RGPD)

Le REAC stipule que le développeur "met en place les mentions légales liées au règlement général sur la protection des données (RGPD)" et "respecte le règlement général sur la protection des données (RGPD) en vigueur". Bien que ce Jalon 2 se concentre sur une API et un client lourd, les principes du RGPD, définis dès le Jalon 1, restent au cœur de l'architecture.

Mon API REST est le principal garant de la protection des données personnelles (identifiants, mots de passe) :

- **Sécurité des accès** : L'authentification par JSON Web Tokens (JWT) assure que seules les personnes authentifiées peuvent accéder aux données.
- **Contrôle des rôles** : J'utilise la gestion des rôles pour garantir que les données personnelles (comme la liste des utilisateurs) ne soient pas exposées. Un utilisateur standard ne peut accéder qu'à ses propres informations, tandis qu'un administrateur a des droits étendus, mais contrôlés.
- **Protection des mots de passe** : Les mots de passe ne sont jamais stockés en clair. Mon service de hachage, validé par des tests unitaires, assure leur non-réversibilité.
- **Minimisation des données** : L'API est conçue pour n'exposer que les données strictement nécessaires à l'action demandée (par exemple, ne pas inclure le mot de passe haché ou l'email dans la réponse d'une consultation de recette).

2. Principes d'Éco-conception (Green IT)

Le REAC m'amène à "identifier les besoins en éco-conception" et à "appliquer les recommandations concernant l'écoconception des services numériques". J'ai appliqué cette démarche à plusieurs niveaux :

- **Optimisation des requêtes** : Dans mon bilan, j'ai noté avoir optimisé des requêtes Entity Framework qui s'avéraient coûteuses. C'est une démarche d'éco-conception : une requête optimisée réduit la charge sur le CPU de la base de données et le temps de traitement, diminuant ainsi la consommation d'énergie.
- **Architecture légère** : Le choix d'une API REST qui communique en JSON est en soi un choix sobre. L'application WinForms ne charge que les données brutes dont elle a besoin, plutôt que de lourdes interfaces graphiques générées côté serveur.
- **Déploiement optimisé** : Mon plan de déploiement repose sur Docker. La conteneurisation est une pratique fondamentale du Green IT : elle permet une mutualisation et une utilisation beaucoup plus dense des ressources d'un serveur physique, réduisant ainsi le nombre de machines nécessaires et l'empreinte matérielle globale.
- **CI/CD** : Mon pipeline de CI/CD exécute les tests à chaque *push*. En interceptant les bugs tôt, j'évite des cycles de déploiement/redéploiement correctifs, qui sont eux-mêmes consommateurs de ressources.

Bilan

Apports et Apprentissages

Ce second jalon a été extrêmement formateur. J'ai pu consolider des compétences techniques pointues :

- La conception d'une architecture en couches propre et la structuration d'un projet .NET.
- La conception, la sécurisation (JWT) et le déploiement d'une API REST.
- La mise en place d'un pipeline de CI/CD complet avec tests automatisés.
- La conteneurisation d'une application multi-services avec Docker et Docker Compose.

J'ai aussi gagné en rigueur sur la gestion de projet, notamment grâce à l'utilisation du tableau Kanban pour planifier et suivre mes tâches.

Difficultés rencontrées

La gestion du temps a également un problème encore maintenant non-résolu.

J'ai dû me familiariser et apprendre à utiliser l'ORM Entity Framework ; j'ai dû notamment ajuster certaines configurations pour optimiser des requêtes qui se sont avérées trop coûteuses en performance.

Perspectives

Pour la suite de mon parcours et pour d'éventuelles améliorations de ce projet, j'aimerais :

- Ajouter l'internationalisation (i18n) et la localisation (l10n) à l'API et à l'application desktop.
- Améliorer l'ergonomie (UX/UI) de l'application WinForms.
- Poursuivre la rédaction de tests pour atteindre une couverture de code (code coverage) de 100%.
- Optimiser le code pour améliorer la scalabilité de l'API.

Remerciements

Au terme de ce dernier jalon de projet, je tiens à exprimer mes sincères remerciements.

Je remercie tout d'abord l'organisme de formation 2iSA pour la qualité de l'enseignement dispensé et les moyens mis à disposition pour notre apprentissage.

Ma gratitude va ensuite tout particulièrement à mes formateurs Serge, Fabien et Théo pour leur accompagnement constant, leur disponibilité et leurs précieux conseils tout au long de cette période, qui ont été déterminants dans ma progression et m'ont encouragé à persévérer.

Je souhaite également remercier mes collègues de promotion pour l'entraide, les échanges constructifs et l'excellente ambiance de travail qui ont grandement contribué à rendre cette expérience de formation agréable et stimulante.

Annexe

[Workflow automatisé des tests de l'API](#)

[Workflow automatisé de la génération d'image docker](#)